

Universidad Siglo 21

Taller de Construcción de Software (INF243)

Período académico 1B – 2026

TRABAJO PRÁCTICO INTEGRADOR

EcoMarket

Plataforma de Comercio Electrónico Sostenible

Informe Técnico Final – Semana 2

Backend (Spring Boot) + Frontend (React)

Integrante: Nahuel Navarro | Legajo: SOF01655

Docente a cargo: Pablo Martin Maldonado

Índice

1. Introducción y alcance del informe final
2. Arquitectura del sistema completo
3. Backend: API REST con Spring Boot
4. Frontend: aplicación React
5. Integración entre frontend y backend
6. Resolución de dificultades encontradas
7. Pruebas y validación end-to-end
8. Uso declarado de herramientas de IA
9. Conclusiones grupales
10. Reflexiones individuales

1. Introducción y alcance del informe final

Este informe documenta la entrega final del trabajo práctico integrador de Taller de Construcción de Software. EcoMarket es una plataforma de comercio electrónico sostenible para una startup argentina dedicada a la venta de productos ecológicos y reutilizables.

La aplicación completa se compone de dos partes desplegadas como aplicaciones independientes que se comunican vía HTTP:

- Un backend con Spring Boot 3.3 que expone una API REST con persistencia JPA sobre MySQL (o H2 en memoria para desarrollo).
- Un frontend con React 18 + Vite que consume la API y provee la interfaz de usuario completa.

Este documento abarca la semana 1 (Backend) y 2 (Frontend), la descripción de la integración entre ambas partes, las dificultades resueltas durante el desarrollo, y la reflexión individual.

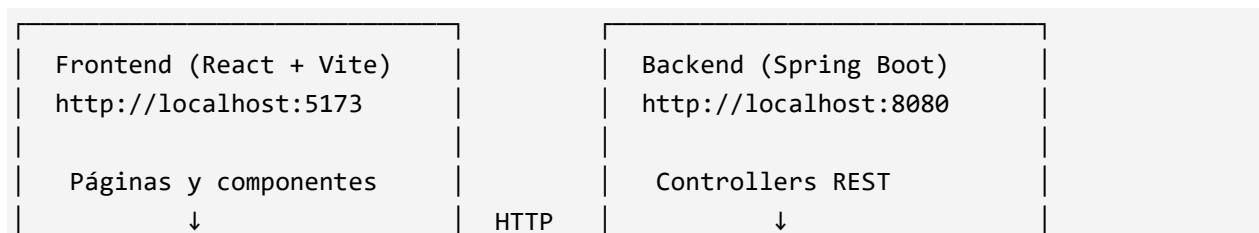
1.1. Estructura del repositorio

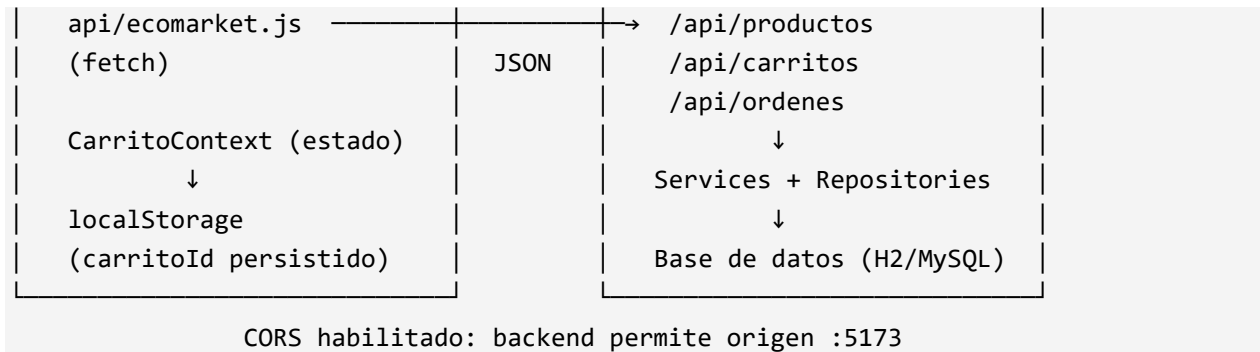
```
Taller-Construccion-NN/
├── backend/           API REST con Spring Boot
│   ├── pom.xml
│   ├── src/main/java/com/ecomarket/...
│   ├── src/main/resources/
│   └── postman/
├── frontend/         Aplicación React + Vite
│   ├── package.json
│   ├── vite.config.js
│   ├── index.html
│   └── src/
└── README.md         Documentación raíz
```

2. Arquitectura del sistema completo

La arquitectura sigue el patrón cliente-servidor con dos aplicaciones que se comunican mediante una API REST sobre HTTP. Cada aplicación tiene su propio ciclo de vida, su propia configuración y se ejecuta en su propio puerto.

2.1. Diagrama de alto nivel





2.2. Stack tecnológico

Aspecto	Backend	Frontend
Lenguaje	Java 17	JavaScript (ES2022) + JSX
Framework	Spring Boot 3.3.4	React 18.3
Build	Maven 3.8+	Vite 5.4
Persistencia	Spring Data JPA + Hibernate	localStorage (carritoid)
Base de datos	MySQL 8 / H2 en memoria	—
Ruteo	Spring MVC	React Router 6
Validación	Jakarta Bean Validation	Validación inline en formularios
Puerto	8080	5173

3. Backend: API REST con Spring Boot

Esta sección resume el backend documentado en detalle en el informe parcial. Para la entrega final no se introdujeron cambios estructurales: la API quedó cerrada en Semana 1 y el frontend se construyó sobre ella sin necesidad de modificar endpoints, DTOs ni reglas de negocio.

3.1. Capas y responsabilidades

- **Controller:** tres controladores REST (ProductoController, CarritoController, OrdenController) que reciben requests HTTP, validan los DTOs con @Valid y delegan al servicio.
- **Service:** lógica de negocio. ProductoService implementa CRUD con soft delete; CarritoService valida stock, agrupa ítems repetidos y calcula totales; OrdenService toma snapshot de precios al confirmar, descuenta stock y marca el carrito como inmutable.

- **Repository:** interfaces Spring Data JPA. Solo agregamos consultas derivadas cuando hicieron falta (findByActivoTrue, findAllByOrderByFechaConfirmacionDesc).
- **Entity:** cinco entidades JPA (Producto, Carrito, ItemCarrito, Orden, ItemOrden) con sus relaciones cascading y orphanRemoval donde correspondía.
- **DTO:** nueve DTOs que actúan como contrato JSON de la API. Se mapean a/desde entidades dentro del service. Esto evita exponer detalles del ORM al frontend.
- **Exception:** GlobalExceptionHandler con @RestControllerAdvice que centraliza todas las respuestas de error en formato ErrorDTO uniforme.

3.2. Endpoints expuestos

La API expone 13 endpoints organizados en tres dominios (productos, carrito, órdenes), todos bajo el prefijo /api. La documentación completa con request/response samples está en el informe parcial. El frontend consume el 100% de estos endpoints sin necesidad de agregar ninguno nuevo.

4. Frontend: aplicación React

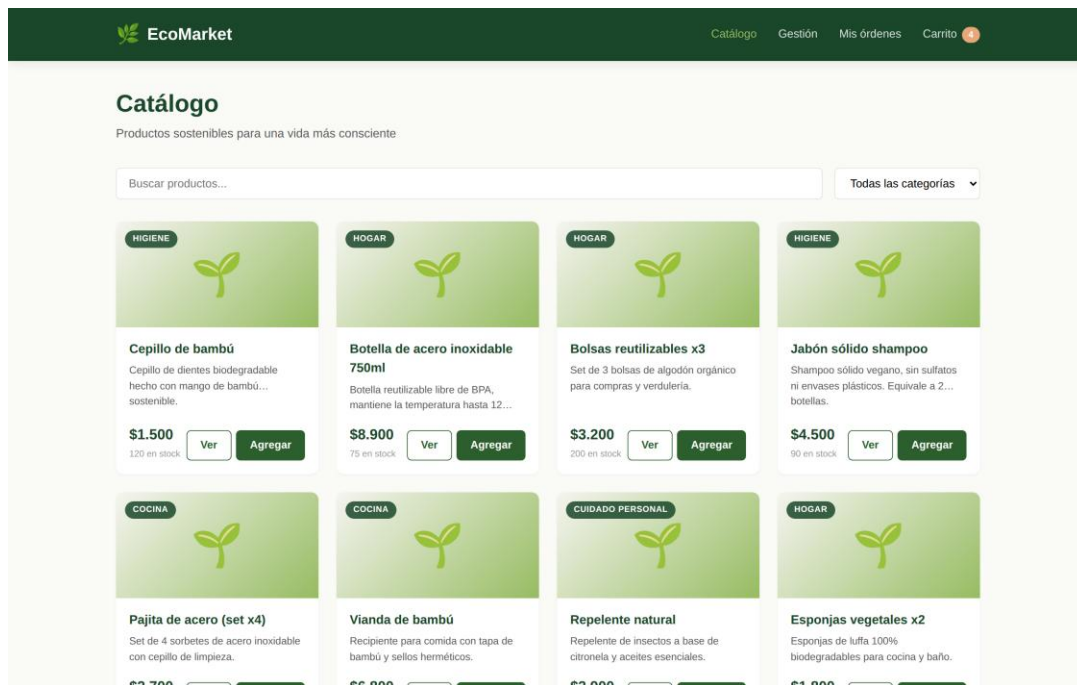
La aplicación frontend se desarrolló con React 18 utilizando exclusivamente componentes funcionales y los hooks useState y useEffect tal como exige el enunciado en la sección 3.2. Vite se eligió como build tool por su velocidad de arranque en desarrollo y por su soporte nativo de JSX y ES Modules sin configuración adicional.

4.1. Estructura de carpetas

frontend/	
├─ package.json	
├─ vite.config.js	
├─ index.html	
├─ public/leaf.svg	Favicon
└─ src/	
├─ main.jsx	Bootstrap React/DOM
├─ App.jsx	Router + providers
├─ styles.css	Estilos globales
├─ api/ecomarket.js	Cliente HTTP único
├─ context/CarritoContext.jsx	Estado global del carrito
├─ components/	
├─ Navbar.jsx	Barra superior + badge
├─ ProductoCard.jsx	Tarjeta de producto
├─ ProductoForm.jsx	Formulario controlado
├─ CarritoItem.jsx	Item del carrito
├─ Modal.jsx	Modal de confirmación
└─ Toast.jsx	Notificaciones

└─ pages/	
└─ CatalogoPage.jsx	Listado público
└─ ProductoDetallePage.jsx	Detalle
└─ AdminProductosPage.jsx	Tabla CRUD
└─ ProductoFormPage.jsx	Crear/Editar
└─ CarritoPage.jsx	Carrito + checkout
└─ OrdenesPage.jsx	Historial
└─ OrdenDetallePage.jsx	Detalle de orden

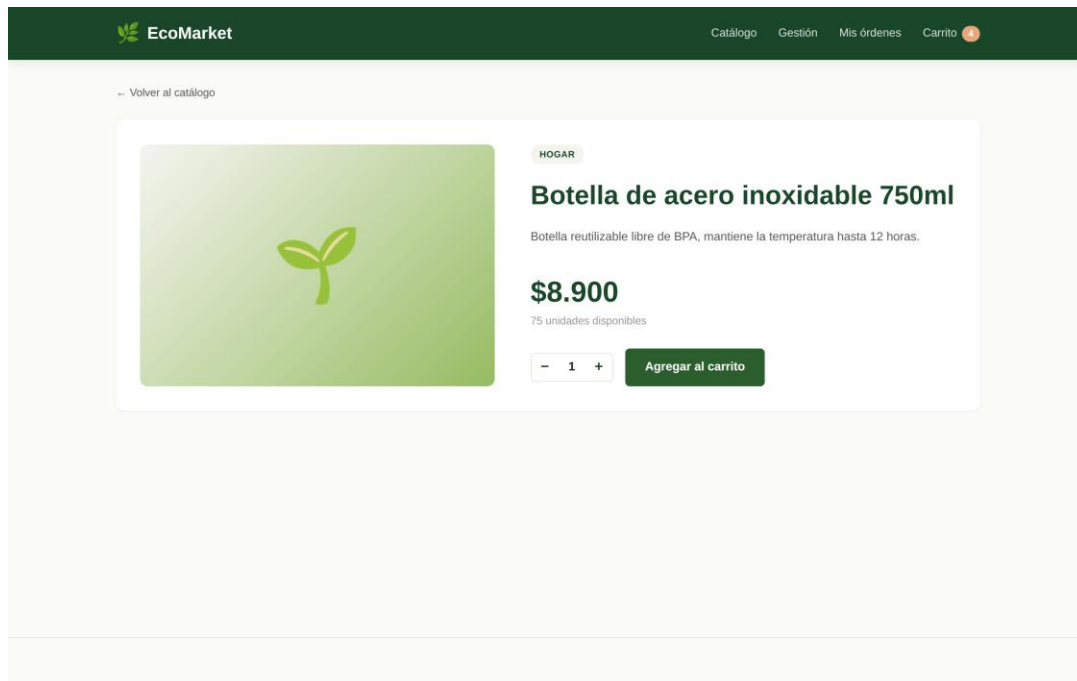
4.2. Catálogo de productos



Catálogo público. Filtros por nombre y categoría. Cada tarjeta tiene los CTAs 'Ver' y 'Agregar'.

La página de catálogo (CatalogoPage) es el punto de entrada de la aplicación. Hace fetch de `/api/productos` al montarse, mantiene en estado local el texto de búsqueda y la categoría seleccionada, y renderiza una grilla responsive de ProductoCard. El componente ProductoCard es un hijo puro que recibe el producto por prop y emite las acciones via callback `onAgregarCarrito`.

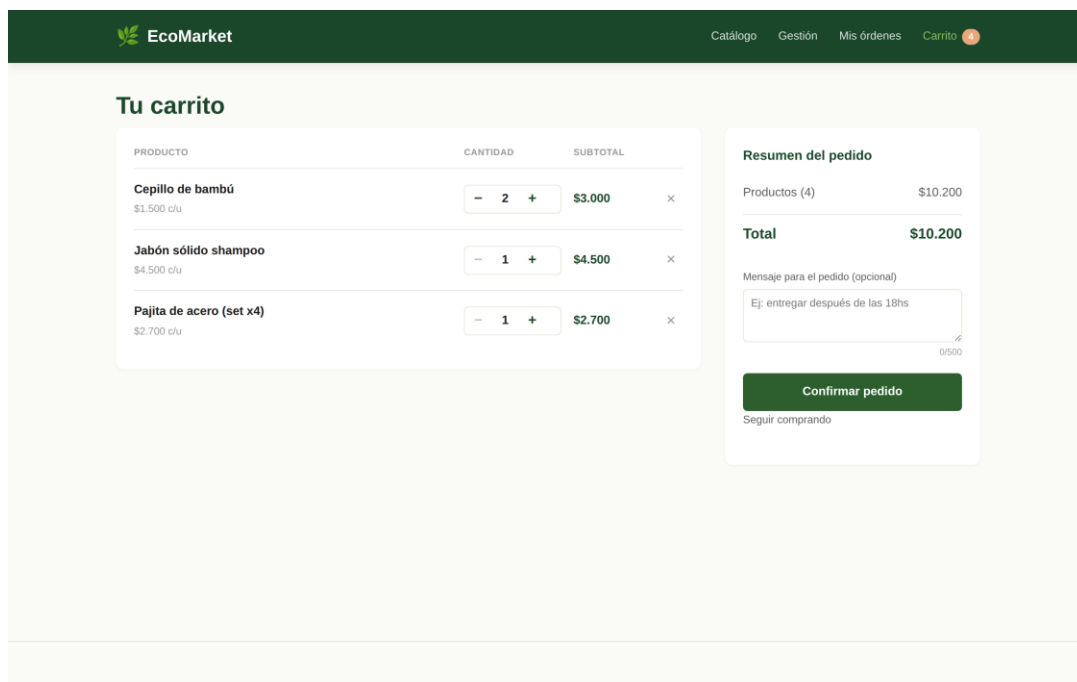
4.3. Detalle de producto



Detalle individual con selector de cantidad y stock disponible.

ProductoDetallePage usa el hook useParams de React Router para leer el id de la URL, hace fetch del producto puntual con useEffect, y permite seleccionar la cantidad antes de agregar al carrito. El selector de cantidad respeta el stock disponible como cota superior.

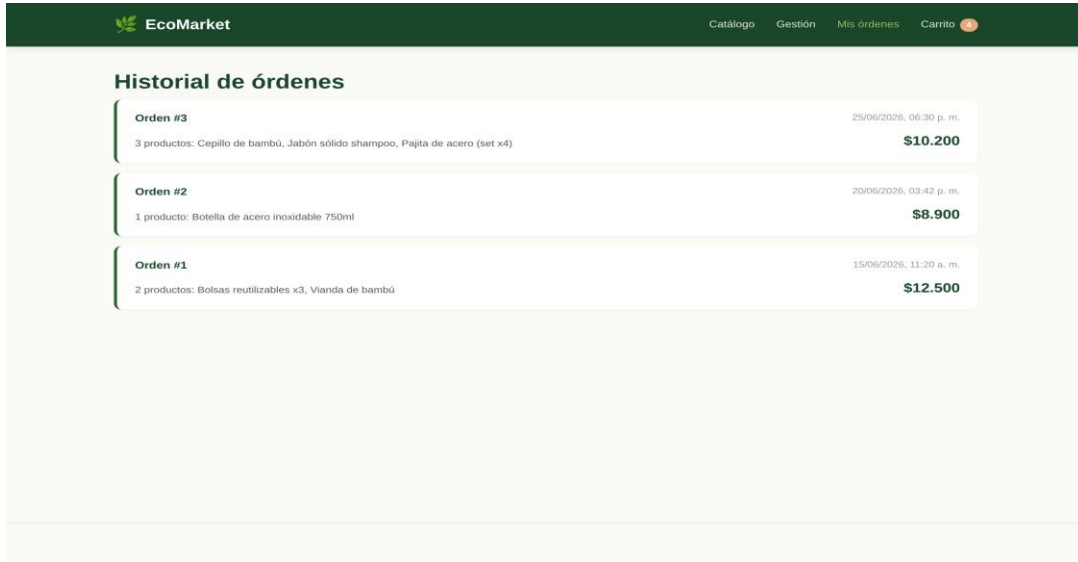
4.4. Carrito de compras



Carrito con tres ítems, total calculado por el backend, mensaje opcional y CTA de confirmación.

CarritoPage es la página más compleja del frontend. Consume el estado del CarritoContext, renderiza una lista de CarritoItem (con selector de cantidad por ítem) y un panel resumen con el total. Los cambios de cantidad y las eliminaciones disparan llamadas al backend, que devuelve el carrito actualizado, y el componente refresca su vista con la respuesta. El campo de mensaje es local con un contador de caracteres (límite 500).

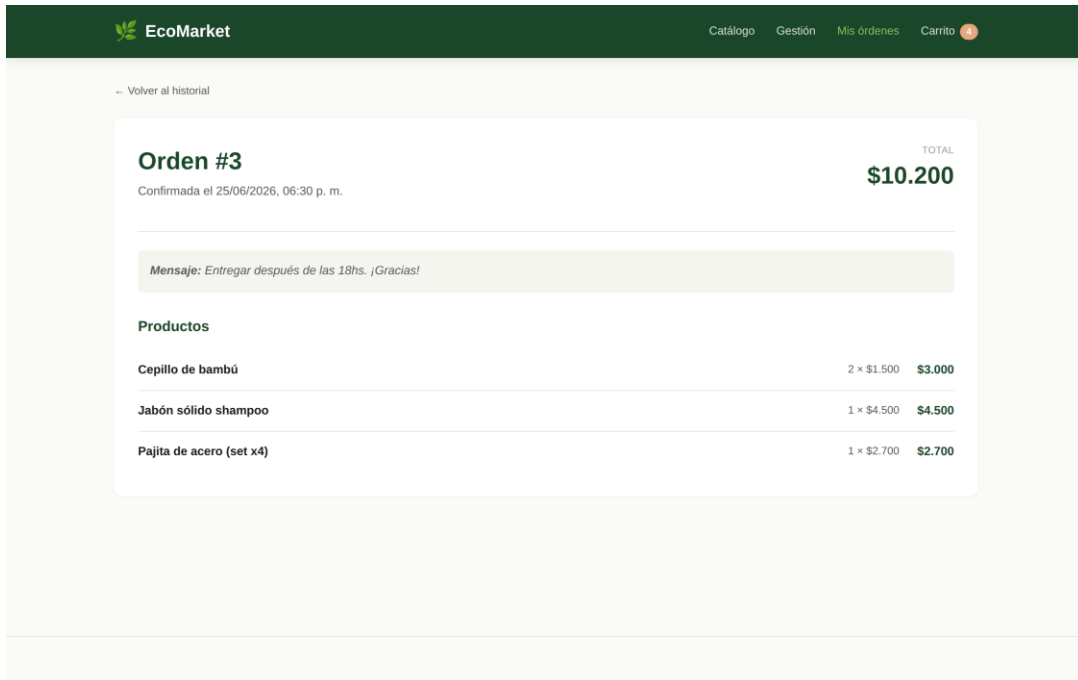
4.5. Historial y detalle de órdenes



Historial de órdenes

Orden	Fecha y hora	Productos	Total
Orden #3	25/06/2026, 06:30 p. m.	3 productos: Cepillo de bambú, Jabón sólido shampoo, Pajita de acero (set x4)	\$10.200
Orden #2	20/06/2026, 03:42 p. m.	1 producto: Botella de acero inoxidable 750ml	\$8.900
Orden #1	15/06/2026, 11:20 a. m.	2 productos: Bolsas reutilizables x3, Vianda de bambú	\$12.500

Historial de órdenes ordenadas de la más reciente a la más antigua.



Orden #3 TOTAL
Confirmada el 25/06/2026, 06:30 p. m. **\$10.200**

Mensaje: Entregar después de las 18hs. ¡Gracias!

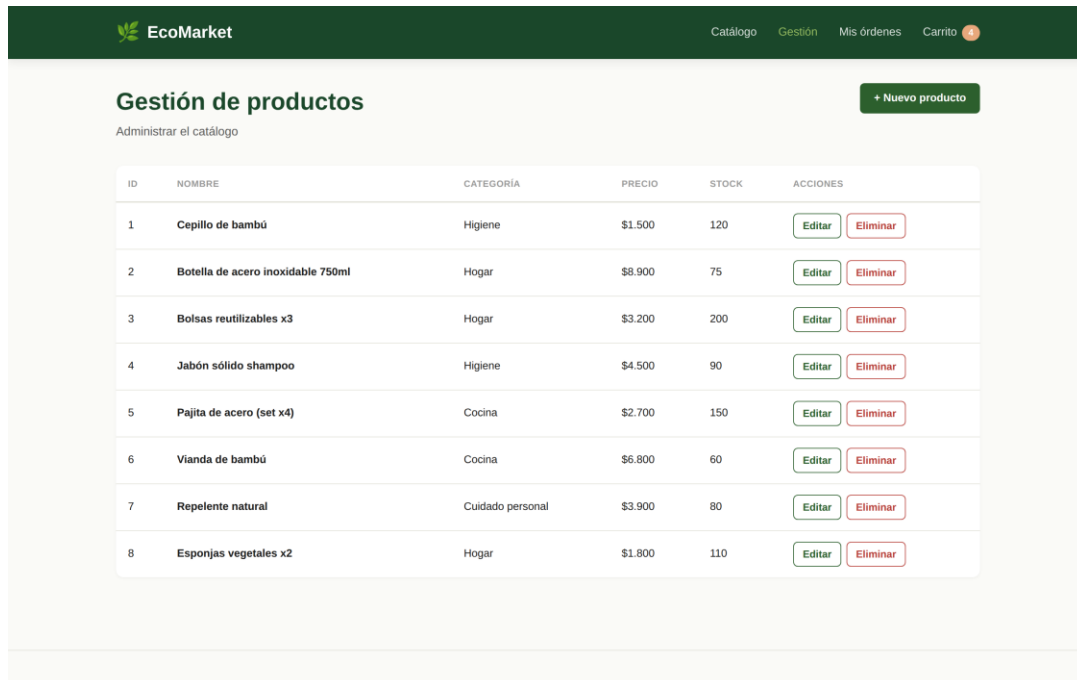
Productos

Producto	Cantidad	Subtotal
Cepillo de bambú	2 × \$1.500	\$3.000
Jabón sólido shampoo	1 × \$4.500	\$4.500
Pajita de acero (set x4)	1 × \$2.700	\$2.700

Detalle de una orden: snapshot inmutable de la compra.

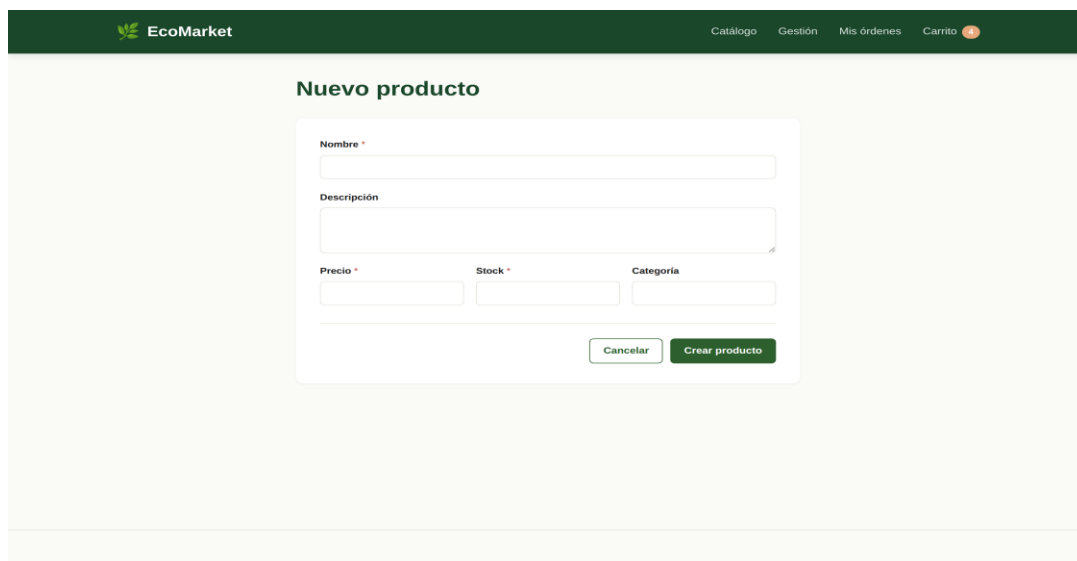
OrdenesPage lista las órdenes ordenadas por fecha de confirmación descendente, con un resumen rápido por cada una. OrdenDetallePage muestra el snapshot inmutable con los nombres y precios congelados en el momento de la compra (decisión documentada en el informe parcial).

4.6. Gestión de productos (CRUD)



ID	NOMBRE	CATEGORÍA	PRECIO	STOCK	ACCIONES
1	Cepillo de bambú	Higiene	\$1.500	120	Editar Eliminar
2	Botella de acero inoxidable 750ml	Hogar	\$8.900	75	Editar Eliminar
3	Boisas reutilizables x3	Hogar	\$3.200	200	Editar Eliminar
4	Jabón sólido shampoo	Higiene	\$4.500	90	Editar Eliminar
5	Pajita de acero (set x4)	Cocina	\$2.700	150	Editar Eliminar
6	Vianda de bambú	Cocina	\$6.800	60	Editar Eliminar
7	Repelente natural	Cuidado personal	\$3.900	80	Editar Eliminar
8	Espojas vegetales x2	Hogar	\$1.800	110	Editar Eliminar

Tabla de gestión con acciones de editar y eliminar (soft delete).



Nuevo producto

Nombre *

Descripción

Precio * Stock * Categoría

[Cancelar](#) [Crear producto](#)

Formulario controlado para crear o editar productos.

AdminProductosPage muestra todos los productos en una tabla con acciones inline. La eliminación abre un Modal de confirmación porque es destructiva. El formulario (ProductoForm) se usa tanto para crear

como para editar: la página padre `ProductoFormPage` detecta si hay un id en la URL para decidir el modo, y le pasa el producto inicial cuando corresponde.

5. Integración entre frontend y backend

La integración entre las dos aplicaciones se resuelve con tres mecanismos: un cliente HTTP único en el frontend, CORS configurado en el backend, y un contrato de errores uniforme. Esto permite que ambas partes evolucionen de forma independiente sin acoplarse fuera de lo estrictamente necesario.

5.1. Cliente HTTP (`api/ecomarket.js`)

Todo el frontend usa un único módulo (`src/api/ecomarket.js`) para hablar con el backend. Internamente expone tres objetos (`productosApi`, `carritoApi`, `ordenesApi`) con un método por endpoint. El módulo envuelve `fetch` y centraliza:

- La URL base del backend (`BASE_URL = 'http://localhost:8080'`).
- Los headers (`Content-Type: application/json`).
- El parseo del cuerpo (`response.json()` o `null` para 204 No Content).
- El manejo de respuestas no exitosas: lanza un `Error` con el mensaje devuelto por el backend y agrega status y detalles.

Cualquier componente que necesite hablar con el backend importa y llama estos métodos. Si en el futuro la URL base cambia o se necesita agregar un token de autenticación, se modifica un solo archivo.

5.2. Flujo de datos unidireccional

Se respetó el patrón solicitado por el enunciado (sección 3.2 del TP): los datos viajan de padres a hijos vía props, y las acciones viajan de hijos a padres vía callbacks. Un ejemplo concreto en `CarritoPage`:

```
function CarritoPage() {
  const { carrito, actualizarCantidad, eliminarItem } = useCarrito();

  const handleActualizar = async (itemId, cantidad) => {
    await actualizarCantidad(itemId, cantidad);
  };

  return carrito.items.map(item => (
    <CarritoItem
      key={item.id}
      item={item} // data hacia abajo
      onActualizarCantidad={handleActualizar} // callback hacia arriba
      onEliminar={handleEliminar} // callback hacia arriba
    />
  ));
}
```

```
));  
}
```

CarritoItem no conoce el contexto ni el cliente HTTP. Solo sabe que recibe un item para mostrar y dos funciones para invocar cuando el usuario interactúa. Esto lo hace fácil de testear y de reusar.

5.3. Estado del carrito

El carrito es el único estado verdaderamente global de la aplicación: lo necesita la Navbar (para mostrar el badge con la cantidad de ítems), el catálogo y el detalle (para agregar productos) y la página del carrito (para ver, modificar y confirmar). Por eso se eligió React Context.

CarritoContext mantiene tres valores: el carritoId, el carrito completo (con sus ítems y total calculado por el backend), y un estado de carga. Las acciones que expone son agregarProducto, actualizarCantidad, eliminarItem y limpiarCarrito.

El carritoId se persiste en localStorage. Esto resuelve un problema real: si el usuario recarga la página, no debería perder los ítems que ya tenía. Al montarse, el provider lee el id de localStorage, intenta refrescar el carrito desde el backend, y si el backend responde 404 o el carrito ya está confirmado, lo descarta y crea uno nuevo cuando haga falta.

5.4. CORS

Por la política same-origin del navegador, sin CORS configurado el navegador bloquearía cualquier request desde localhost:5173 hacia localhost:8080. El backend resuelve esto con la clase com.ecomarket.config.CorsConfig:

```
@Configuration  
public class CorsConfig implements WebMvcConfigurer {  
    @Override  
    public void addCorsMappings(CorsRegistry registry) {  
        registry.addMapping("/api/**")  
            .allowedOrigins(  
                "http://localhost:5173",  
                "http://localhost:3000",  
                "http://127.0.0.1:5173",  
                "http://127.0.0.1:3000"  
            )  
            .allowedMethods("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS")  
            .allowedHeaders("*")  
            .allowCredentials(true);  
    }  
}
```

5.5. Manejo de errores unificado

El backend nunca devuelve HTML ni stack traces. Toda excepción se intercepta en `GlobalExceptionHandler` y se serializa como un `ErrorDTO` con timestamp, status, error, mensaje y detalles. El frontend, en el wrapper `request()` del cliente HTTP, captura cualquier respuesta no exitosa y lanza un `Error` con esos campos. Las páginas atrapan el error y muestran un toast al usuario, con la posibilidad de ver detalles cuando vienen en el array detalles.

6. Resolución de dificultades encontradas

Durante el desarrollo de Semana 2 surgieron varios problemas que requirieron decisiones de diseño o ajustes de configuración. Se documentan a continuación con su contexto, la solución aplicada y la justificación.

6.1. CORS bloqueando requests desde el navegador

Al levantar el frontend por primera vez y hacer la primera llamada a `/api/productos`, el navegador bloqueó la respuesta con el clásico mensaje 'has been blocked by CORS policy'. Esto era esperable: el frontend corre en `localhost:5173` y el backend en `localhost:8080`, lo que el navegador considera orígenes distintos. La solución, ya prevista desde Semana 1, fue la clase `CorsConfig` que habilita explícitamente el origen del frontend. La política de aceptar también `localhost:3000` y las versiones con 127.0.0.1 cubre cualquier variante de cómo se levante el frontend en distintas máquinas.

6.2. Persistencia del carrito entre recargas

El carrito en el backend está identificado por un id que devuelve `POST /api/carritos`. Si el frontend no recordara ese id, una recarga de página haría que el usuario perdiera todos sus ítems. Se evaluaron tres opciones:

- Crear el carrito en cada render: descartado porque perdería los ítems en cada recarga.
- Mantener el id en memoria (`useState` a nivel App): descartado porque no sobrevive a F5.
- Persistir el id en `localStorage`: elegida. Sobrevive a recargas y al cierre del navegador.

Se agregó una protección contra inconsistencias: al cargar el id desde `localStorage`, el provider intenta refrescar el carrito desde el backend. Si el backend responde 404 (carrito borrado) o el carrito ya está confirmado, el frontend lo descarta y crea uno nuevo cuando haga falta.

6.3. Lombok incompatible con JDK reciente

Al intentar compilar el backend con `mvn spring-boot:run` desde la terminal de Windows, Maven falló con 96 errores de 'cannot find symbol' sobre todos los métodos generados por Lombok (getters, setters, builder, constructor). La compilación previa desde VS Code había funcionado porque la extensión Java de Red Hat usa Eclipse JDT con su propia integración Lombok, distinto de la pipeline real de Maven.

El problema tenía dos causas combinadas:

- **Maven Compiler Plugin 3.11+** (que viene con Spring Boot 3.3.4) dejó de detectar annotation processors automáticamente del classpath. Hay que declararlos explícitamente en annotationProcessorPaths.
- **Lombok 1.18.34** (versión que Spring Boot 3.3.4 maneja por defecto) solo soporta hasta JDK 23. En máquinas con JDK 24+ instalado en el PATH, Lombok rompe con 'TypeTag :: UNKNOWN'.

La solución fue actualizar el pom.xml con dos cambios concretos:

```
<properties>
  <java.version>17</java.version>
  <lombok.version>1.18.46</lombok.version>
</properties>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <annotationProcessorPaths>
      <path>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
```

Fijando Lombok 1.18.46 (la versión estable a abril de 2026 que soporta JDK 24/25/26) y declarando el annotationProcessorPath, los 96 errores se resolvieron y el backend compiló correctamente.

6.4. Race condition al refrescar el carrito en navegación rápida

La primera versión de las páginas tenía useEffect que disparaba fetch sin cleanup. Si el usuario navegaba rápidamente entre páginas, llegaba a darse el caso donde una respuesta vieja sobrescribía a una más reciente. Se solucionó con el patrón estándar de bandera cancelada en useEffect:

```
useEffect(() => {
  let cancelado = false;
  productosApi.listar()
    .then(data => { if (!cancelado) setProductos(data); })
    .catch(err => { if (!cancelado) setError(err.message); });
  return () => { cancelado = true; };
}, []);
```

Este patrón se aplicó en todas las páginas que hacen fetch.

6.5. Conflicto entre id del carrito y carrito ya confirmado

Si el usuario confirmaba una orden y luego volvía atrás, el localStorage seguía teniendo el id del carrito viejo (ahora confirmado). El backend respondía con el carrito marcado como confirmado=true, pero el frontend no debía permitir agregar más ítems a un carrito confirmado. La solución fue triple:

- El backend valida en CarritoService (verificarEditable) y devuelve 409 Conflict si se intenta modificar un carrito confirmado.
- El frontend, al refrescar el carrito en el provider, detecta la bandera confirmado=true y descarta el id automáticamente.
- Al confirmar la orden con éxito, el frontend llama limpiarCarrito() para borrar el id de localStorage proactivamente.

7. Pruebas y validación end-to-end

Con la aplicación completa funcionando, se ejecutaron pruebas manuales del flujo completo del usuario y de los casos de error más representativos.

7.1. Flujo happy path

- 1. Abrir <http://localhost:5173>. El catálogo carga los 8 productos seed.
- 2. Filtrar por categoría 'Higiene'. La lista se reduce a 2 productos.
- 3. Agregar tres productos distintos al carrito. El badge de la navbar se actualiza a 3.
- 4. Entrar al carrito. Cambiar la cantidad del primer ítem a 2. El total se recalcula.
- 5. Escribir un mensaje opcional y confirmar el pedido.
- 6. El frontend redirecciona al detalle de la orden. Se ve el total y los ítems con sus precios congelados.
- 7. Volver a 'Mis órdenes'. La nueva orden aparece primera en el historial.
- 8. Volver al catálogo. Los productos comprados tienen su stock descontado.

7.2. Casos de error manejados

- **Stock insuficiente:** agregar más unidades de las disponibles dispara un toast rojo con el mensaje del backend.
- **Producto inexistente:** navegar a </productos/999> muestra el error 404 del backend en pantalla.
- **Validación de formulario:** intentar crear un producto con precio 0 muestra el mensaje inline al lado del campo antes de enviar.
- **Backend caído:** si el backend no está corriendo, las páginas muestran 'Error: Failed to fetch' en su zona de error.

- **Carrito confirmado:** intentar agregar a un carrito ya confirmado dispara 409 Conflict; el frontend lo detecta y crea uno nuevo.

8. Uso declarado de herramientas de IA

Conforme al punto 7 del enunciado del trabajo práctico, se declara explícitamente el uso de herramientas de inteligencia artificial generativa durante el desarrollo de esta entrega.

Herramienta utilizada

Claude (Anthropic), modelo conversacional de propósito general.

Partes del trabajo en las que se utilizó

- **Backend:** asistente para debuggear archivos Java (errores de compilación y excepciones en tiempo de ejecución) y tareas puntuales de configuración de Spring Boot (perfiles de aplicación, CORS, propiedades de JPA, integración Lombok con Maven).
- **Frontend:** asistente para debuggear archivos JSX (errores de hooks, dependencias de useEffect, sintaxis) y tareas puntuales de configuración de Vite y React Router.

Verificación:

Todo el código fue revisado, ajustado y probado. Las decisiones de diseño (snapshot de precios, soft delete, separación DTO/entidad, flujo unidireccional, manejo del estado del carrito en localStorage, patrón de cleanup en useEffect) están documentadas y pueden ser explicadas y defendidas durante la defensa oral.

9. Conclusiones

La construcción de EcoMarket como aplicación fullstack en dos sprints de una semana cada uno me interiorizo sobre la importancia del orden en el desarrollo: tener el backend cerrado y probado al final de Semana 1 hizo que Semana 2 fuera mucho menos ansiosa. El frontend se construyó contra un contrato estable, y el único trabajo de integración fue conectar el cliente HTTP a los endpoints ya documentados.

Las decisiones de diseño tomadas en Semana 1 (snapshot de precios, soft delete, separación DTO/entidad) demostraron su valor al armar el frontend. El snapshot de precios, en particular, me permitió mostrar el historial de órdenes con datos consistentes aunque modifique productos en paralelo desde la página de gestión. La separación de DTOs me dio un contrato JSON claro contra el cual diseñar las pantallas, sin tener que entender el modelo interno de JPA.

Las dificultades concretas que enfrenté (CORS, persistencia del carrito, Lombok incompatible con JDK reciente, race conditions en useEffect) son ejemplos del tipo de problemas que aparecen al integrar dos aplicaciones. Resolverlas implicó entender no solo mi propio código sino también el comportamiento de

las herramientas que usé. La experiencia más tediosa fue el debugging de Lombok: me obligó a leer la documentación de Maven Compiler Plugin, entender cómo funcionan los annotation processors, y descubrir que el comportamiento que veía en VS Code no era el mismo que el de la línea de comandos.

La aplicación final cumple con todos los requisitos del enunciado: cubre los tres bloques funcionales (catálogo, carrito, órdenes), respeta el patrón unidireccional de React, usa hooks funcionales, integra ambas partes con CORS resuelto, y queda documentada en este informe y en los READMEs de cada subproyecto.

10. Reflexión personal

A continuación comparto mi reflexión personal sobre el proceso de desarrollo de EcoMarket: qué aprendí, qué haría diferente y cuáles fueron los desafíos más formativos del proyecto.

Navarro, Nahuel

Qué aprendí

Este proyecto me permitió profundizar en la construcción de una aplicación fullstack real de punta a punta. En el backend, consolidé mi comprensión del patrón de capas de Spring Boot (controller → service → repository → entity) y entendí en la práctica por qué la separación de DTOs y entidades JPA es importante: me dio un contrato JSON estable contra el cual construir todo el frontend sin que los detalles internos de Hibernate se filtraran a la interfaz. La implementación de OrdenService, con el snapshot de precios y el descuento atómico de stock, fue donde más aprendí sobre lógica transaccional y sobre cómo las decisiones de modelado impactan en la consistencia de los datos.

Qué haría diferente

Dedicaría más tiempo a pruebas automatizadas desde el principio. Toda la validación fue manual con Postman y con el navegador, lo cual funcionó para el alcance del TP pero no escala. Agregaría tests unitarios en los services del backend (al menos para las reglas de stock, el soft delete y el snapshot de precios) y tests de componente en React para los formularios controlados y el CarritoContext. También evaluaría usar TypeScript en el frontend para tener chequeo de tipos en tiempo de compilación, especialmente en los DTOs que viajan desde el backend.

Desafíos más formativos

El desafío más formativo fue resolver la incompatibilidad de Lombok con JDK 24+. Los 96 errores de compilación me obligaron a investigar cómo funcionan los annotation processors en Maven, por qué el maven-compiler-plugin 3.11+ dejó de detectarlos automáticamente del classpath, y por qué la versión de Lombok que maneja Spring Boot 3.3.4 no soportaba mi JDK. La solución (fijar lombok.version a 1.18.46 y declarar el annotationProcessorPath) fue técnicamente simple, pero el proceso de diagnóstico me enseñó

a leer logs de Maven con atención y a no confiar en que el comportamiento del IDE refleja la pipeline real de build. El segundo desafío fue diseñar el estado del carrito en el frontend: elegir entre useState local, Context, o un state manager externo, y decidir qué persistir en localStorage y qué delegar al backend. La solución final (Context + localStorage del carritoId + revalidación contra el backend al montar) me dejó una comprensión más sólida de cómo manejar estado en aplicaciones React que dependen de un servidor.